

Guía técnica de despliegue y ejecución

El presente documento tiene como finalidad proporcionar las instrucciones precisas para aprovisionar el entorno de laboratorio utilizado en este Trabajo Final de Grado. Siguiendo estos pasos, se podrá desplegar el clúster de Kubernetes, la infraestructura de observabilidad (OpenTelemetry, Jaeger, OpenSearch y Prometheus) y la aplicación de microservicios (Spring Petclinic), así como ejecutar los cuatro escenarios prácticos de validación.

1. Entorno de referencia empleado

Para la construcción y ejecución empírica de este laboratorio, se ha empleado una máquina virtual configurada con las siguientes características:

- **Sistema Operativo:** Ubuntu 22.04 LTS.
- **Recursos:** 14 vCPUs y 24 GB de memoria RAM.
- **Privilegios:** Usuario con permisos de administración (sudo).

Nota: La arquitectura y las herramientas utilizadas son altamente flexibles. El ecosistema podría llegar a desplegarse y funcionar sobre otras distribuciones de GNU/Linux o con una asignación de memoria y procesador inferior. No obstante, las especificaciones aquí descritas son las empleadas durante el estudio y garantizan la estabilidad total del sistema.

2. Preparación del entorno base

2.1. Instalación de software requerido

Antes de comenzar con el despliegue de la infraestructura, es necesario disponer de ciertas herramientas de red, control de versiones y, fundamentalmente, el kit de desarrollo de Java (JDK) en su versión 17 para poder compilar posteriormente los microservicios.

Bash

```
# Actualización de repositorios e instalación de paquetes básicos y Java 17
sudo apt update
sudo apt install curl git openjdk-17-jdk -y

# Configuración de la variable de entorno JAVA_HOME
echo "export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64" >> ~/.bashrc
source ~/.bashrc

# Comprobación de que la versión instalada es correcta
java -version
```

2.2. Instalación de K3s

El laboratorio utiliza K3s como motor de orquestación de Kubernetes. Se configurará para ejecutarse de forma ligera en un único nodo y permitir la lectura del archivo de configuración sin necesidad de utilizar el usuario `root`.

Bash

```
# Descarga e instalación de K3s definiendo el nombre del nodo
curl -sL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -s - --node-name control1

# Tras unos segundos, comprobamos que el nodo está en estado Ready
kubectl get nodes
```

Para asegurar que las herramientas externas puedan comunicarse con el clúster correctamente, copiamos el archivo `kubeconfig` al directorio personal del usuario:

Bash

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
export KUBECONFIG=~/.kube/config
```

2.3. Instalación de Helm

Helm es la herramienta estándar para la gestión de paquetes en Kubernetes y resulta indispensable para aprovisionar componentes del laboratorio que vamos a utilizar.

Bash

```
# Descarga y ejecución del script de instalación oficial de Helm
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh

# Confirmamos la correcta instalación
helm list -A
```

2.4. Preparación de docker y container registry local

Para construir las imágenes personalizadas de la aplicación Spring Petclinic y almacenarlas de forma accesible para el clúster, se requiere la instalación de Docker y la configuración de un registro de contenedores local.

Instalación de Docker:

Bash

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
```

Despliegue del Registro Local:

Creamos un directorio en la máquina para garantizar la persistencia de las imágenes y levantamos el contenedor oficial del registro, exponiéndolo en el puerto 5000.

Bash

```
mkdir -p ~/data-container-reg

docker run -d \
  -p 5000:5000 \
  --restart=always \
  --name container-registry-local \
  -v ~/data-container-reg:/var/lib/registry \
  registry:2
```

Configuración de K3s para aceptar el registro local: Por defecto, Kubernetes exige conexiones seguras (HTTPS) para la descarga de imágenes. Al operar nuestro registro local en HTTP, es necesario configurar una excepción en K3s.

Bash

```
sudo mkdir -p /etc/rancher/k3s

# Creamos el fichero de configuración indicando el registro inseguro
sudo tee /etc/rancher/k3s/registries.yaml > /dev/null <<EOF
mirrors:
  "localhost:5000":
    endpoint:
      - "http://localhost:5000"
EOF

# Reiniciamos el servicio de K3s para aplicar la nueva política
sudo systemctl restart k3s
```

3. Despliegue del laboratorio

Una vez configurada la infraestructura base y aseguradas las dependencias, procederemos a descargar el código del proyecto y a ejecutar los scripts de automatización encargados de levantar el entorno al completo.

3.1. Clonación del repositorio

Todo el código fuente, incluyendo la versión adaptada de la aplicación Spring Petclinic, los manifiestos de Kubernetes y los scripts de operación, se encuentra alojado en un repositorio centralizado. Para descargarlo y acceder a su directorio de trabajo, ejecutamos:

Bash

```
git clone https://github.com/cvgmsys/spring-petclinic-opentelemetry-demo.git
cd spring-petclinic-opentelemetry-demo
```

3.2. Compilación y publicación de imágenes

El siguiente paso consiste en preparar los microservicios para su despliegue en el clúster. Para ello, se ha diseñado un script que automatiza el proceso completo: utiliza Maven para compilar el código de Java, construye las imágenes de Docker de los distintos componentes y las publica en el registro de contenedores local (`localhost:5000`) bajo la etiqueta `latest`.

Bash

```
./scripts/build_and_push.sh
```

3.3. Despliegue del *stack* de observabilidad

Antes de lanzar la aplicación de negocio, es necesario establecer la capa de monitorización y trazabilidad. El siguiente script se encarga de aprovisionar todo el ecosistema de observabilidad en el clúster. Esto incluye la instalación de prerequisites como `cert-manager`, seguido del despliegue del Operador de OpenTelemetry, Jaeger, OpenSearch y Prometheus.

Bash

```
./scripts/deploy_observability.sh
```

3.4. Despliegue de la aplicación de demostración

Finalmente, procedemos a desplegar la aplicación de demostración. Este script aplica los manifiestos de Kubernetes que levantan el *API Gateway*, los tres microservicios principales (Clientes, Veterinarios y Visitas) y sus respectivas bases de datos MySQL. Gracias a la configuración del entorno, el Operador de OpenTelemetry detectará las anotaciones en los manifiestos e inyectará de forma automática y transparente el agente de instrumentación en las máquinas virtuales de Java durante el arranque de los contenedores.

Bash

```
./scripts/deploy_application.sh
```

4. Reproducción de los escenarios prácticos

Con el entorno completamente desplegado y operativo, procederemos a la ejecución de los cuatro escenarios empíricos. En esta sección se detallan las acciones y comandos necesarios para simular las condiciones de cada prueba.

4.1. Escenario 1: Visibilidad transaccional

Este escenario valida la auto-instrumentación y la correcta propagación del contexto distribuido bajo condiciones de uso normal.

Generación de tráfico manual:

Acceder a la interfaz de usuario de Spring Petclinic a través del navegador web utilizando la ruta local y navegar hasta la ficha de un cliente específico:
`http://localhost:30080/owners/6`.

Validación en Jaeger:

Acceder a la consola de interfaz de Jaeger, expuesto en: `http://localhost:30686`. En el panel de búsqueda, seleccionar el servicio `api-gateway` y buscar las trazas recientes. Se debe localizar la traza correspondiente a la ruta navegada y verificar en su diagrama de cascada la interacción entre el API Gateway, el servicio de clientes y la base de datos, así como la captura automática de metadatos (como el código HTTP 200).

4.2. Escenario 2: Cuello de botella

Simulación de un episodio de saturación de CPU para contrastar la ambigüedad de las métricas clásicas frente a la precisión del análisis de trazas.

Inyección de tráfico:

Generar carga base y, paralelamente, lanzar el script de estrés dirigido a saturar específicamente el servicio de veterinarios:

Bash

```
k6 run scripts/baseline_load_test.js &  
k6 run scripts/stress_test.js
```

Comprobación de lentitud:

Recargar en el navegador la ruta `http://localhost:30080/vets` y observar el incremento del tiempo de carga (> 2 segundos).

Análisis de métricas en Prometheus:

Consultar en Prometheus, expuesto en la ruta `http://localhost:30090`, el consumo de CPU para evidenciar la subida simultánea en varios contenedores, lo que genera ambigüedad diagnóstica:

PromQL

```
sum(rate(container_cpu_usage_seconds_total{namespace="spring-petclinic"}[2m])) by (pod)
```

Análisis de trazas en Jaeger:

Localizar en Jaeger, expuesto en: `http://localhost:30086`, la traza afectada. Observar el diagrama de cascada para identificar los grandes tramos de inactividad (estrangulamiento de CPU) en el *span* de *vets-service*, mientras que las consultas a la base de datos se resuelven en milisegundos.

4.3. Escenario 3: Fallo en cascada

Provocación de una interrupción de red interna simulando una pérdida de conectividad de un microservicio con su base de datos.

Generación de tráfico base:

Injectar tráfico normal.

Bash

```
k6 run scripts/baseline_load_test.js
```

Provocación del fallo:

Alterar el selector del *Service* de Kubernetes para desvincular la base de datos de clientes y reiniciar el pod afectado:

Bash

```
kubectl patch service customers-db -n spring-petclinic \  
-p '{"spec":{"selector":{"app.kubernetes.io/instance":"customers-db-rotta"}}}'  
kubectl delete pod customers-db-0 -n spring-petclinic
```

Comprobación del error:

Acceder a `http://localhost:30080/owners/1` y confirmar la recepción de un Error 500. Localizar la traza en Jaeger y verificar que intercepta la excepción nativa de Java (Connection Refused).

Restauración del sistema:

Revertir el parcheo del servicio y reiniciar los pods dependientes para recuperar la normalidad:

Bash

```
kubectl patch service customers-db -n spring-petclinic \
  -p '{"spec":{"selector":{"app.kubernetes.io/instance":"customers-db"}}}'
kubectl delete pod -n spring-petclinic -l app=customers-service
kubectl delete pod -n spring-petclinic -l app=api-gateway
```

4.4. Escenario 4: Impacto operativo

Medición comparativa del consumo de recursos de la aplicación Spring Petclinic con y sin la auto-instrumentación.

Generación de tráfico base:

Inyectar tráfico normal durante unos minutos.

Bash

```
k6 run scripts/baseline_load_test.js
```

Retirada de la auto-instrumentación:

Editar los Deployments de la aplicación (customers-service, vets-service, visits-service y api-gateway) y comentar la anotación: `instrumentation.opentelemetry.io/inject-java: "true"`

Generar tráfico:

Reiniciar los pods de la aplicación para aplicar los cambios y lanzar de nuevo el tráfico de K6 (baseline_load_test.js) de forma sostenida (unos 15-20 minutos).

Medición mediante PromQL:

Acceder a Prometheus, expuesto en la ruta `http://localhost:30090`, y ejecutar las siguientes consultas para extraer los datos comparativos en ambos estados (instrumentado y sin instrumentar):

PromQL

```
# Para medir el uso de CPU
sum(rate(container_cpu_usage_seconds_total{namespace="spring-petclinic",
pod!~"(.*)-db-(.*)", container!="POD", container!=""}[4m])) by (pod) * 1000
```

```
# Para medir el uso de Memoria RAM
sum(container_memory_working_set_bytes{namespace="spring-petclinic", pod!~"(.*)-db-(.*)",
container!="POD", container!=""}) by (pod) / (1024 * 1024)

# Para medir el tráfico de red interno
sum(rate(container_network_transmit_bytes_total{namespace="spring-petclinic",
pod!~"(.*)-db-(.*)"}[4m])) by (pod) / 1024
```

5. Borrado del entorno

Una vez finalizadas las pruebas y la experimentación, se recomienda liberar los recursos de la máquina ejecutando los procesos de limpieza automatizados. Estos comandos eliminan los recursos de Kubernetes y depuran el almacenamiento local de contenedores.

Bash

```
# 1. Eliminación de la aplicación de negocio
./scripts/delete_application.sh

# 2. Eliminación del stack de observabilidad
./scripts/delete_observability.sh

# 3. Borrado de todas las imágenes cacheadas en Docker local
docker rmi -f $(docker images -aq)

# 4. Limpieza de datos en el Container Registry local
./scripts/container_registry.sh clean
```